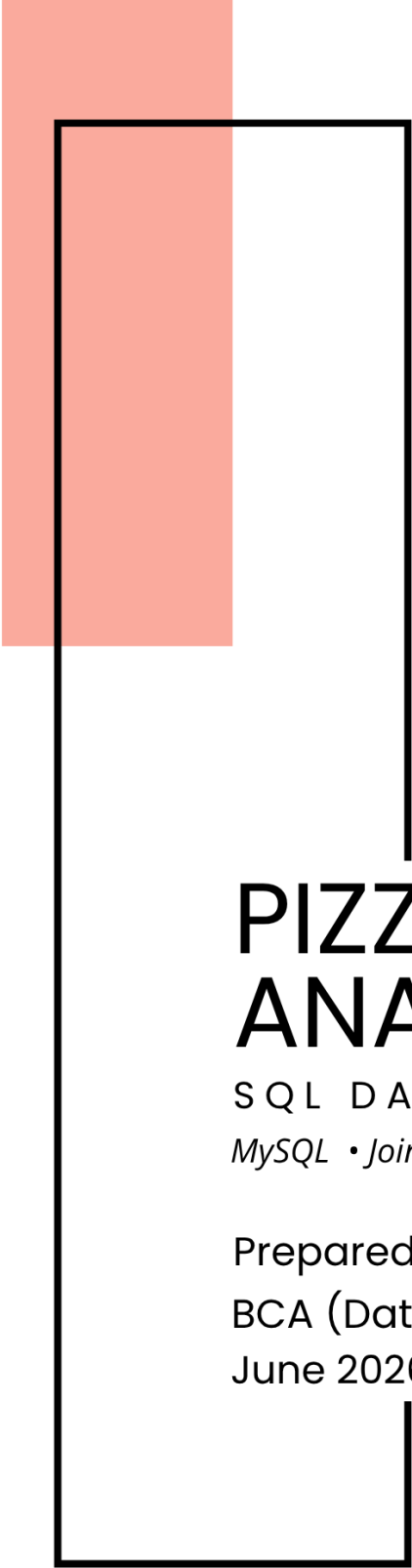




PIZZA SALES ANALYSIS

SQL DATA ANALYTICS PROJECT

MySQL • Joins • CTEs • Window Functions

Prepared by Unnati Singh

BCA (Data Science & AI)

June 2026

Table of Contents

1. Introduction	3
2. Project Objectives	4
3. Tools & Technologies used	4
4. Dataset Overview	5
5. Database Schema	8
6. Analytical Questions & Solutions	9
7. SQL Concepts Demonstrated	23
8. Key Business Insights	24
9. Conclusion	25

INTRODUCTION

The Pizza Sales Analysis project aims to analyze customer ordering patterns and business performance using SQL. Various analytical queries were written to extract insights regarding sales, revenue, customer preferences, and product performance.

The project demonstrates practical SQL skills including joins, aggregations, grouping, subqueries, Common Table Expressions (CTEs), and window functions.

Project Objectives

The objectives of this project are:

- Analyze total orders and revenue.
- Identify popular pizza sizes and categories.
- Understand ordering patterns by hour and date.
- Determine top-performing pizzas.
- Measure revenue contribution by each pizza.
- Analyze cumulative revenue over time.
- Generate business insights for decision-making.

Tools & Technologies Used

- MySQL
- SQL
- MySQL Workbench
- Microsoft Word
- Canva

Dataset Overview

The project uses four tables:

The Pizza Sales dataset contains transactional information related to customer orders, pizza details, and product categories. The dataset is organized into four relational tables that are connected through primary and foreign keys.

The data enables analysis of customer ordering behavior, revenue generation, product popularity, and category performance.

Table 1: orders

This table stores information about each order placed by customers.

Column Name	Description
-------------	-------------

order_id	Unique identifier for each order
order_date	Date on which the order was placed
order_time	Time at which the order was placed

Table 2: order_details

This table contains the details of pizzas included in each order.

Column Name	Description
-------------	-------------

order_details_id	Unique identifier for each order item
------------------	---------------------------------------

order_id	References the order placed
----------	-----------------------------

pizza_id	References the pizza ordered
----------	------------------------------

quantity	Number of pizzas ordered
----------	--------------------------

Table 3: pizzas

This table stores information about pizza sizes and prices.

Column Name	Description
-------------	-------------

pizza_id	Unique identifier for each pizza
----------	----------------------------------

pizza_type_id	References the pizza type
---------------	---------------------------

size	Size of the pizza (S, M, L, XL, XXL)
------	--------------------------------------

price	Price of the pizza
-------	--------------------

Table 4: pizza_types

This table contains information about pizza names, categories, and ingredients.

Column Name Description

pizza_type_id	Unique identifier for each pizza type
name	Name of the pizza
category	Pizza category (Classic, Chicken, Veggie, Supreme)
ingredients	Ingredients used in the pizza

The relational structure of these tables allows complex analysis through SQL joins, enabling the extraction of meaningful business insights related to sales performance and customer preferences.

Database Schema

orders

|

| order_id

↓

order_details

|

| pizza_id

↓

pizzas

|

| pizza_type_id

↓

pizza_types

Analytical Questions Addressed

1. Retrieve the total number of orders placed.

Purpose:

Determine overall customer activity.

Solution:

```
1 • SELECT
2     COUNT(order_id) AS total_orders
3 FROM
4     orders;
5
```

Result Grid				Filter Rows:	Export: 	Wrap Cell Content: 
	COUNT(order_id)					
▶	21350					

2. Calculate the total revenue generated from pizza sales.

Purpose:

Measure business performance.

Solution:

```
1 • SELECT
2     ROUND(SUM(od.quantity * p.price), 2) AS total_revenue
3 FROM
4     pizzas p
5     INNER JOIN
6     order_details od ON p.pizza_id = od.pizza_id;
```

Result Grid			 Filter Rows:	Export: 	Wrap Cell Content: 
	total_revenue				
▶	817860.05				

3. Identify the highest-priced pizza.

Purpose:

Find premium products.

Solution:

```
1 • SELECT
2     pT.name, p.price
3 FROM
4     pizzas p
5     JOIN
6     pizza_types pT ON pT.pizza_type_id = p.pizza_type_id
7 WHERE
8     price = (SELECT
9             MAX(price)
10            FROM
11            pizzas);
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	name	price			
▶	The Greek Pizza	35.95			

4. Identify the most common pizza size ordered.

Purpose:

Understand customer preferences.

Solution:

```
1 • SELECT
2     p.size, SUM(od.quantity) AS total_orders
3 FROM
4     order_details od
5     JOIN
6     pizzas p ON od.pizza_id = p.pizza_id
7 GROUP BY p.size
8 ORDER BY total_orders DESC
9 LIMIT 1;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:	Fetch rows:
	size	total_orders				
▶	L	18956				

5. Top 5 most ordered pizza types along with their quantities.

Purpose:

Identify best-selling products.

Solution:

```
1 • SELECT
2     pT.name, SUM(od.quantity) AS total_quantity
3 FROM
4     pizza_types pT
5     JOIN
6     pizzas p ON pT.pizza_type_id = p.pizza_type_id
7     JOIN
8     order_details od ON p.pizza_id = od.pizza_id
9 GROUP BY pT.pizza_type_id , pT.name
10 ORDER BY total_quantity DESC
11 LIMIT 5;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:	Fetch rows:
	name	total_quantity				
▶	The Classic Deluxe Pizza	2453				
	The Barbecue Chicken Pizza	2432				
	The Hawaiian Pizza	2422				
	The Pepperoni Pizza	2418				
	The Thai Chicken Pizza	2371				

6. Join the necessary tables to find the total quantity of each pizza category ordered.

Purpose:

Evaluate category demand.

Solution:

```
1 • SELECT
2     pT.category, SUM(od.quantity) AS total_quantity
3 FROM
4     order_details od
5     JOIN
6     pizzas p ON od.pizza_id = p.pizza_id
7     JOIN
8     pizza_types pT ON p.pizza_type_id = pT.pizza_type_id
9 GROUP BY pT.category;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	category	total_quantity			
▶	Classic	14888			
	Veggie	11649			
	Supreme	11987			
	Chicken	11050			

7. Determine the distribution of orders by hour of the day.

Purpose:

Determine peak ordering periods.

Solution:

```
1 • SELECT
2     HOUR(order_time) AS hour, COUNT(*) AS total_orders
3 FROM
4     orders
5 GROUP BY hour
6 ORDER BY hour;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	hour	total_orders			
▶	9	1			
	10	8			
	11	1231			
	12	2520			
	13	2455			
	14	1472			
	15	1468			
	16	1920			
	17	2336			
	18	2399			
	19	2009			
	20	1642			
	21	1198			
	22	663			
	23	28			

8. Join relevant tables to find the category-wise distribution of pizzas.

Purpose:

Compare category popularity.

Solution:

```
1 • SELECT
2     category, COUNT(name) AS total_pizzas
3 FROM
4     pizza_types
5 GROUP BY category;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	category	total_pizzas			
▶	Chicken	6			
	Classic	8			
	Supreme	9			
	Veggie	9			

9. Group the orders by date and calculate the average number of pizzas ordered per day.

Purpose:

Analyze daily demand.

Solution:

```
1 • SELECT
2     ROUND(AVG(total_pizzas), 2) AS avg_pizzas_per_day
3 FROM
4     (SELECT
5         o.order_date, SUM(od.quantity) AS total_pizzas
6     FROM
7         orders o
8     LEFT JOIN order_details od ON o.order_id = od.order_id
9     GROUP BY o.order_date) AS t;
```

Result Grid			Filter Rows:	Export: 	Wrap Cell Content: 
	avg_pizzas_per_day				
▶	138.47				

10. Determine the top 3 most ordered pizza types based on revenue.

Purpose:

Identify high-value products.

Solution:

```
1 • SELECT
2     p.pizza_type_id,
3     pT.name,
4     SUM(od.quantity * p.price) AS revenue
5 FROM
6     pizza_types pT
7     JOIN
8     pizzas p ON p.pizza_type_id = pT.pizza_type_id
9     JOIN
10    order_details od ON p.pizza_id = od.pizza_id
11 GROUP BY p.pizza_type_id , pT.name
12 ORDER BY revenue DESC
13 LIMIT 3;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:	Fetch rows:
	pizza_type_id	name	revenue		
▶	thai_chn	The Thai Chicken Pizza	43434.25		
	bbq_chn	The Barbecue Chicken Pizza	42768		
	cali_chn	The California Chicken Pizza	41409.5		

11. Calculate the percentage contribution of each pizza type to total revenue.

Purpose:

Understand revenue drivers.

Solution:

```
1 • SELECT category,  
2     ROUND(revenue/SUM(revenue)OVER () * 100 ,2) AS contribution_in_revenue  
3 FROM( SELECT category,  
4        SUM(od.quantity * p.price) AS revenue  
5 FROM  
6     pizza_types pT  
7     JOIN  
8     pizzas p ON p.pizza_type_id = pT.pizza_type_id  
9     JOIN  
10    order_details od ON p.pizza_id = od.pizza_id  
11 GROUP BY pT.category) AS t;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	category	contribution_in_revenue			
▶	Classic	26.91			
	Veggie	23.68			
	Supreme	25.46			
	Chicken	23.96			

12. Analyze the cumulative revenue generated over time.

Purpose:

Track business growth trends.

Solution:

```
1 • SELECT
2     order_date,
3     revenue_per_day,
4     SUM(revenue_per_day) OVER (ORDER BY order_date) AS cumulative_revenue
5 FROM
6 (
7     SELECT
8         o.order_date,
9         ROUND(SUM(od.quantity * p.price)) AS revenue_per_day
10    FROM pizzas p
11   JOIN order_details od
12     ON p.pizza_id = od.pizza_id
13   JOIN orders o
14     ON od.order_id = o.order_id
15   GROUP BY o.order_date
16 ) AS t
17 ORDER BY order_date;
```

order_date	revenue_per_day	cumulative_revenue
2015-01-01	2714	2714
2015-01-02	2732	5446
2015-01-03	2662	8108
2015-01-04	1755	9863
2015-01-05	2066	11929
2015-01-06	2429	14358
2015-01-07	2202	16560
2015-01-08	2838	19398
2015-01-09	2127	21525
2015-01-10	2464	23989
2015-01-11	1872	25861
2015-01-12	1919	27780

13. Determine the top 3 most ordered pizza types based on revenue for each pizza category.

Purpose:

Identify category leaders.

Solution:

```
1  ● ○ WITH ranked_pizzas AS (  
2      SELECT  
3          pt.category,  
4          pt.name,  
5          ROUND(SUM(od.quantity * p.price)) AS revenue,  
6          RANK() OVER (  
7              PARTITION BY pt.category  
8              ORDER BY SUM(od.quantity * p.price) DESC  
9          ) AS rnk  
10     FROM pizza_types pt  
11     JOIN pizzas p  
12         ON pt.pizza_type_id = p.pizza_type_id  
13     JOIN order_details od  
14         ON p.pizza_id = od.pizza_id  
15     GROUP BY  
16         pt.category,  
17         pt.name  
18 )  
19     SELECT  
20         name,  
21         category,  
22         revenue  
23     FROM ranked_pizzas  
24     WHERE rnk <= 3  
25     ORDER BY  
26         category,  
27         revenue DESC;
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



	name	category	revenue
►	The Thai Chicken Pizza	Chicken	43434
	The Barbecue Chicken Pizza	Chicken	42768
	The California Chicken Pizza	Chicken	41410
	The Classic Deluxe Pizza	Classic	38180
	The Hawaiian Pizza	Classic	32273
	The Pepperoni Pizza	Classic	30162
	The Spicy Italian Pizza	Supreme	34831
	The Italian Supreme Pizza	Supreme	33477
	The Sicilian Pizza	Supreme	30940
	The Four Cheese Pizza	Veggie	32266
	The Mexicana Pizza	Veggie	26781
	The Five Cheese Pizza	Veggie	26066

SQL Concepts Demonstrated

Joins

- INNER JOIN
- LEFT JOIN

Aggregate Functions

- SUM()
- AVG()
- COUNT()
- ROUND()

Grouping

- GROUP BY
- ORDER BY

Date and Time Functions

- HOUR()
- DATE()

Window Functions

- SUM() OVER()
- DENSE_RANK()

Subqueries

Nested queries for advanced analysis.

KEY INSIGHTS

- The analysis helped identify the total number of orders placed and the overall revenue generated from pizza sales.
- Customer ordering preferences were explored by analyzing the most commonly ordered pizza sizes and pizza types.
- Category-wise sales analysis provided an understanding of which pizza categories received the highest demand.
- Hourly order distribution helped identify peak ordering periods and customer purchasing patterns throughout the day.
- Revenue-based analysis highlighted the pizzas that contributed the most to overall sales performance.
- Percentage contribution analysis was used to understand the share of total revenue generated by different pizza types.
- Cumulative revenue analysis demonstrated how revenue accumulated over time and introduced the use of SQL window functions for trend analysis.
- Category-level ranking analysis helped identify the top-performing pizzas within each pizza category.
- The project demonstrated how SQL can be used to transform raw transactional data into meaningful business information.
- Through this project, practical experience was gained in writing joins, aggregations, subqueries, Common Table Expressions (CTEs), and window functions to solve real-world business problems.

Conclusion

This Pizza Sales Analysis project was completed using SQL to analyze customer orders, product performance, and revenue trends. The project involved working with multiple related tables and applying various SQL concepts such as joins, aggregate functions, subqueries, CTEs, and window functions.

The analysis helped answer important business questions related to sales performance, customer preferences, product popularity, and revenue contribution. Through this project, valuable hands-on experience was gained in querying relational databases and extracting meaningful insights from data.

Overall, the project strengthened practical SQL skills and provided experience in solving business-oriented analytical problems using real-world datasets.